

Lưu ý dành cho các giáo viên và người dùng khác của những slide này: Chúng tôi rất vui nếu bạn thấy tài liệu của chúng tôi hữu ích cho việc giảng dạy của mình. Bạn có thể sử dụng các slide này nguyên văn hoặc chỉnh sửa chúng cho phù hợp với nhu cầu của riêng mình. Nếu bạn sử dụng một phần đáng kể các slide này trong bài giảng của mình, vui lòng ghi kèm thông báo này hoặc liên kết đến trang web của chúng tôi: <http://www.mm>

Frequent Itemset Mining & Association Rules

CS246: Khai thác các tập dữ liệu khổng lồ
Jure Leskovec, Đại học Stanford
<http://cs246.stanford.edu>



Announcements

Hôm nay chúng tôi đã phát bài tập về nhà số 1. Hạn nộp là 2 tuần (22/1 lúc 23:59). Vui lòng bắt đầu làm sớm. Bài tập khá dài. Yêu cầu chứng minh định lý cũng như lập trình.

Chúng tôi đã đăng tải Colab 0 và Colab 1. Hạn nộp là 1 tuần nữa (15/1 lúc 23:59).

Gửi thư OAE đến địa chỉ email của khóa

học cs246-win2526-staff@lists.stanford.edu Tất

cả các liên kết HW/Colab sẽ được đăng trên Ed.

Announcements

Các buổi ôn tập về Spark, Đại số tuyến tính và Xác suất.

Bản ghi âm sẽ được đăng tải trên Canvas

Giờ làm việc

Bắt đầu vào tuần tới

Association Rule Discovery

Quản lý kệ hàng siêu thị -

Mô hình giỏ hàng:

Mục tiêu: Xác định các mặt hàng được mua cùng nhau bởi đủ số lượng khách hàng

Phương pháp: Xử lý dữ liệu bán hàng thu thập được bằng máy quét mã vạch để tìm ra mối liên hệ giữa các mặt hàng.

Một quy tắc kinh điển:

Nếu ai đó mua tã và sữa, thì người đó là
có khả năng mua bia

Đừng ngạc nhiên nếu bạn thấy các loại bia đóng gói sẵn đặt cạnh tã bỉm!

The Market-Basket Model

Một tập hợp lớn các **mặt hàng**

Ví dụ: những mặt hàng được

bán trong siêu thị.

Một bộ **giỏ** lớn

Mỗi giỏ hàng là

một tập hợp nhỏ các mặt hàng.

Ví dụ: những thứ mà

một khách hàng mua trong một ngày.

Khám phá các quy tắc liên kết:

Những người đã mua $\{x, y, z\}$ thường có xu hướng mua $\{v, w\}$.

Ví dụ ứng dụng: Amazon

Đầu vào:

Các mặt hàng trong giỏ hàng	
1	Bánh mì, Coca-Cola, Sữa
2	Bia, bánh mì
3	Bia, Coca-Cola, Tã bỉm, Sữa
4	Bia, bánh mì, tã lót, sữa
5	Coca-Cola, Tã bỉm, Sữa

Kết quả đầu ra:

Các quy tắc đã được phát hiện:

Sữa --> Coca-Cola

Tã bỉm, Sữa --> Bia

More generally

Một ánh xạ nhiều-nhiều tổng quát
(sự liên kết) giữa hai loại sự vật

Nhưng chúng ta quan tâm đến mối liên hệ giữa các “mặt hàng”,
chứ không phải giữa các “giỏ hàng”.

Các vật phẩm và giỏ hàng là những khái niệm trừu tượng:

Ví dụ:

Các mặt hàng/giỏ hàng có thể là sản phẩm/giỏ mua sắm

Các mục/giỏ có thể là từ ngữ/tài liệu

Các mục/giỏ có thể là các cặp bazơ/gen

Các vật phẩm/giỏ hàng có thể là thuốc/bệnh nhân

Applications – (1)

Sản phẩm = hàng hóa; **Giỏ hàng** = tập hợp các sản phẩm mà ai đó đã mua trong một lần đi mua sắm

Giỏ hàng thực tế: Các chuỗi cửa hàng lưu trữ hàng TB dữ liệu về những mặt hàng khách hàng thường mua cùng nhau.

Cho biết cách khách hàng điển hình di chuyển trong cửa hàng, cho phép họ sắp xếp các mặt hàng hấp dẫn cạnh nhau:

Câu chuyện không có thật về phát hiện “tã bỉm và bia”

Được sử dụng để đặt khoai tây chiên giữa tã bỉm và bia nhằm thúc đẩy doanh số bán khoai tây chiên.

Chính sách "những người mua X cũng mua Y" của Amazon

Applications – (2)

Giỏ = câu; **Mục** = tài liệu trong
những câu đó xuất hiện

Các mục xuất hiện cùng nhau quá thường xuyên có thể
là dấu hiệu của đạo văn.

Lưu ý rằng các mặt hàng không nhất thiết phải được "đặt" trong giỏ hàng.

Giỏ = bệnh nhân; **Vật phẩm** = thuốc & tác dụng phụ

Đã được sử dụng để phát hiện các tổ hợp
của các loại thuốc gây ra các tác dụng phụ cụ thể

Nhưng cần mở rộng: Cần quan sát cả sự vắng mặt của một
vật phẩm cũng như sự hiện diện của nó.

Outline

Đầu tiên: Định nghĩa

Các tập hợp mục thường xuyên

Quy tắc của hội:

Sự tự tin, sự hỗ trợ, sự thú vị

Tiếp theo: Các thuật toán tìm tập hợp mục thường xuyên

Tìm các cặp thường xuyên

Thuật toán A-Priori

Thuật toán PCY

Frequent Itemsets

Câu hỏi đơn giản nhất: Tìm các nhóm đồ vật thường xuyên xuất hiện cùng nhau trong giỏ hàng.

Hỗ trợ cho tập hợp mặt hàng I: Số lượng giỏ hàng

bao gồm tất cả các mục trong I

(Thường được biểu thị dưới dạng phân số của tổng số giỏ hàng)

Cho trước ngưỡng hỗ trợ s ,

Khi đó, các tập hợp mặt hàng xuất hiện trong ít nhất s giỏ hàng được gọi là các

tập hợp mặt hàng thường xuyên.

Các mục	TID
1	Bánh mì, Coca-Cola, Sữa
2	Bia, bánh mì
3	Bia, Coca-Cola, Tã bỉm, Sữa
4	Bia, bánh mì, tã lót, sữa
5	Coca-Cola, Tã bỉm, Sữa

Hỗ trợ của
 $\{\text{Bia, Bánh mì}\} = 2$

Example: Frequent Itemsets

Các mặt hàng = {sữa, coca, pepsi, bia, nước ép}

Ngưỡng hỗ trợ = 3 giỏ hàng

B1 = {m, c, b}

B2 = {m, p, j}

B3 = {m, b}

B4 = {c, j}

B5 = {m, p, b}

B6 = {m, c, b, j}

B7 = {c, b, j}

B8 = {b, c}

Các tập hợp mục thường xuyên xuất hiện: {m}, {c}, {b}, {j},
{m,b} , {b,c} , {c,j}.

Define: Association Rules

Định nghĩa: Luật liên kết:

Các quy tắc điều kiện "nếu-thì" về nội dung của các giỏ hàng.

$\{i_1, i_2, \dots, i_k\} \rightarrow \{j\}$ có nghĩa là: "nếu một giỏ chứa tất cả các $i_1, \dots, i_k$ thì nó có khả năng chứa $\{j\}$ "

Trên thực tế

có rất nhiều quy tắc, chúng ta muốn tìm những quy tắc quan trọng/thú

vị! Độ tin cậy của quy tắc liên kết

là xác suất của j khi biết $I = \{i_1, \dots, i_k\}$ support()

$$\text{conf}(I \rightarrow j) = \frac{\text{hỗ trợ}(I \rightarrow j)}{\text{hỗ trợ}(I)}$$

$$\text{conf}(I \rightarrow j) = \frac{\text{support}(I \rightarrow j)}{\text{support}(I)}$$

Interesting Association Rules

Không phải tất cả các quy tắc có độ tin cậy cao đều thú vị

Quy tắc X → sữa có thể có độ tin cậy cao đối với nhiều tập hợp mặt hàng X, bởi vì sữa được mua rất thường xuyên (độc lập với X)

Lãi suất của một quy tắc liên kết I → j:

sự khác biệt tuyệt đối giữa độ tin cậy của nó và tỷ lệ các giỏ chứa j

$$(Tôỉ) = | ()_P = | [] | Các (|) () |$$

quy tắc thú vị: những quy tắc có giá trị lãi suất cao (thường trên 0,5)

Tại sao lại dùng giá trị tuyệt đối? Muốn nắm bắt cả giá trị dương và âm.

và mối liên hệ tiêu cực giữa các tập hợp mục và các mục

Example: Confidence and Interest

$$B1 = \{m, c, b\}$$

$$B2 = \{m, p, j\}$$

$$B3 = \{m, b\}$$

$$B4 = \{c, j\}$$

$$B5 = \{m, p, b\}$$

$$B6 = \{m, c, b, j\}$$

$$B7 = \{c, b, j\}$$

$$B8 = \{b, c\}$$

Quy tắc liên kết: $\{m, b\}$ c

Hỗ trợ = 2

Độ tin cậy = $2/4 = 0,5$

Lãi suất = $|0,5 - 5/8| = 1/8$

Mặt hàng c xuất hiện trong 5/8 số giỏ hàng

Luật này không thú vị lắm!

Association Rule Mining

Bài toán: Tìm tất cả các quy tắc liên kết có độ
hỗ trợ $\geq s$ và độ tin cậy $\geq c$ Lưu ý:

Độ hỗ trợ của một quy tắc liên kết là độ hỗ trợ của toàn bộ
tập hợp các mục trong quy tắc đó (vế trái + vế phải)

Phần khó: Tìm các tập hợp mục thường xuyên! Nếu

$\{i_1, i_2, \dots, i_k\} \cup \{j\}$ có độ hỗ trợ và độ tin
cậy cao, thì cả $\{i_1, i_2, \dots, i_k\}$ và $\{i_1, i_2, \dots, i_k, j\}$
đều sẽ là “thường xuyên”

$$\text{conf}(\text{Tôi } j) = \frac{\text{hỗ trợ}(\text{Tôi } j)}{\text{hỗ trợ}(\text{Tôi } j)}$$

Mining Association Rules

$$\text{conf}(I \rightarrow j) = \frac{\text{support}(I \cup j)}{\text{support}(I)}$$

Bước 1: Tìm tất cả các tập hợp mục thường xuyên I

(Chúng tôi sẽ giải thích điều này ở phần tiếp theo)

Bước 2: Tạo luật

Với mỗi tập con A của I , hãy tạo ra một quy tắc $A \rightarrow I \setminus A$

Vì I xuất hiện thường xuyên, nên A cũng xuất hiện thường xuyên.

Biến thể 1: Chỉ thực hiện một lần tính toán độ tin cậy của quy tắc

$$\text{độ tin cậy}(A, B \rightarrow C, D) = \text{sự ủng hộ}(A, B, C, D) / \text{sự ủng hộ}(A, B)$$

Biến thể 2:

Quan sát: Nếu $A, B, C \rightarrow D$ nằm dưới mức độ tin cậy, thì $A, B \rightarrow C, D$ cũng vậy.

Có thể tạo ra các quy tắc "lớn hơn" từ các quy tắc nhỏ hơn!

Xuất ra các quy tắc vượt quá ngưỡng tin cậy

Example

B1 = {m, c, b}

B2 = {m, p, j}

B3 = {m, c, b, n}

B4 = {c, j}

B5 = {m, p, b}

B6 = {m, c, b, j}

B7 = {c, b, j}

B8 = {b, c}

Ngưỡng hỗ trợ $s = 3$, độ tin cậy $c = 0,75$

Bước 1) Tìm các tập hợp mục thường xuyên xuất hiện:

{b,m} {b,c} {c,m} {c,j} {m,c,b}

Bước 2) Tạo ra các quy tắc:

~~b m: c=4/6~~ b c: c=5/6 ~~b,c m: c=3/5~~

m b: c=4/5 ~~b,m c: c=3/4~~

~~b c,m: c=3/6~~

Compacting the Output

Để giảm số lượng quy tắc, chúng ta có thể xử lý hậu kỳ chúng và chỉ xuất ra:

Tập hợp các mục thường xuyên tối đa:

Không có tập hợp con trực tiếp nào thường xuyên xuất hiện.

Giúp tẩy càn nhiều hơn

hoặc

Tập hợp mục đóng:

Không có tập hợp con nào ngay lập tức có cùng mức hỗ trợ (> 0)

Lưu trữ không chỉ thông tin thường xuyên mà còn cả thông tin chính xác.

hỗ trợ/số lượng

Example: Maximal/Closed

Hỗ trợ tối đa ($s=3$) Đúng

A 4

KHÔNG

KHÔNG

B 5

KHÔNG

Đúng

C 3

KHÔNG

KHÔNG

AB 4

Đúng

Đúng

AC 2

KHÔNG

KHÔNG

BC 3

Đúng

Đúng

ABC 2

KHÔNG

Đúng

Thường xuyên, nhưng

tập hợp con BC

Cũng thường xuyên.

Thường xuyên và

tập hợp con duy nhất của nó,

ABC, không phải tần suất.

Tập hợp con BC

Có sự hỗ trợ tương tự.

Tập hợp con duy

nhất của nó, ABC, có

Hỗ trợ nhỏ hơn.

Step 2: Finding Frequent Itemsets

Computation Model

Chi phí thực sự của việc khai thác dữ liệu thường trú trên đĩa thường là số lượng thao tác I/O trên đĩa.

Trên thực tế, luật liên kết
các thuật toán đọc dữ liệu theo từng lượt.
- tất cả các giở đều được đọc lần lượt

Chúng tôi đo lường chi phí bằng số lần thuật toán xử lý dữ liệu.

Mục
Mục
Mục
Mục
Mục
Mục
Mục
Mục
Mục
Mục
Mục
Mục
Vân vân.

Các mục là các số nguyên dương, và khoảng cách giữa các giỏ hàng là -1.

Main-Memory Bottleneck

Đối với nhiều thuật toán tìm tập hợp mục thường xuyên, bộ nhớ chính là tài nguyên quan trọng.

Khi đọc các giỏ hàng, chúng ta cần đếm.

một cái gì đó, ví dụ như số lần xuất hiện của các cặp vật phẩm.

Số lượng các vật thể khác nhau mà chúng ta có thể đếm được bị giới hạn bởi bộ nhớ chính.

Việc hoán đổi số lượng bộ nhớ trong/ngoài bộ nhớ chính là một điều không tốt.
ý tưởng

Hỏi: Tại sao?

Finding Frequent Pairs

Vấn đề khó khăn nhất thường là tìm ra **các cặp** mục $\{i_1, i_2\}$ xuất hiện thường xuyên nhất.

Tại sao? Các cặp số thường gặp, các bộ ba số thường gặp.

Tại sao? Xác suất xuất hiện thường xuyên giảm theo cấp số mũ khi kích thước tăng; số lượng tập hợp tăng chậm hơn khi kích thước tăng.

Trước tiên, chúng ta hãy tập trung vào các cặp, sau đó mở rộng sang các tập hợp lớn hơn.

Finding Frequent Pairs

Phương pháp tiếp cận:

Chúng ta luôn cần phải “tạo ra” tất cả các tập hợp mục.

Nhưng chúng tôi chỉ muốn đếm (theo dõi) mà thôi.

những tập hợp mục đó cuối cùng lại trở nên thường xuyên.

Tình huống:

Giả sử mục tiêu của chúng ta là xác định các cặp thường xuyên xuất hiện.

Chúng ta cần liệt kê tất cả các cặp vật phẩm.

Với mỗi giỏ hàng, hãy liệt kê tất cả các cặp mặt hàng trong giỏ hàng đó.

Nhưng thay vì đếm từng cặp, chúng tôi hy vọng sẽ loại bỏ nhiều cặp và chỉ theo dõi những cặp cuối cùng sẽ xuất hiện thường xuyên.

Naïve Algorithm

Cách tiếp cận đơn giản để tìm các cặp thường xuyên xuất hiện Đọc tệp một lần, đếm số lần xuất hiện của mỗi cặp trong bộ

nhớ chính: Từ mỗi giỏ b gồm n_b mặt hàng, tạo ra $n_b (n_b - 1)/2$ cặp bằng hai vòng lặp lồng nhau

Một cấu trúc dữ liệu sẽ đếm số lượng của mỗi cặp.

Thất bại nếu $(\text{\#items})^2$ vượt quá bộ nhớ chính Lưu ý:

$\text{\#items}$ có thể là

1 triệu (Wal-Mart) hoặc 10 tỷ (trang

web) Giả sử có 10^6 mặt hàng, số lượng là

các số nguyên 4 byte Số cặp mặt hàng: 10^6

$(10^6 - 1)/2 \approx 5 \cdot 10^{11}$ Do đó, cần $2 \cdot 10^{12}$ (2 terabyte) bộ nhớ

Counting Pairs in Memory

Mục tiêu: Đếm số lần xuất hiện của mỗi cặp mục (i, j) :

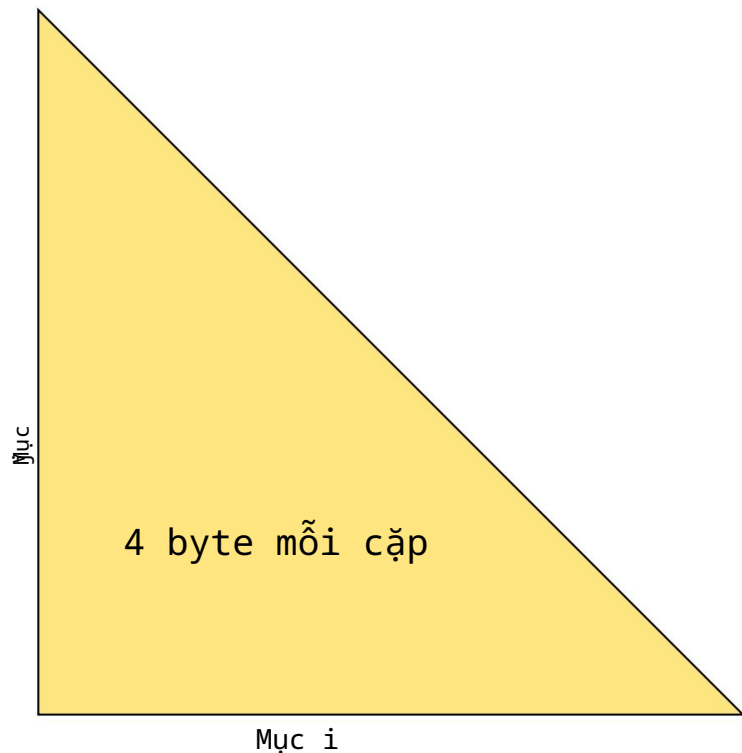
Phương pháp 1: Đếm tất cả các cặp bằng cách sử dụng ma trận

Cách tiếp cận 2: Lập bảng các bộ ba $[i, j, c]$ = “số lượng của cặp phần tử $\{i, j\}$ là c .”

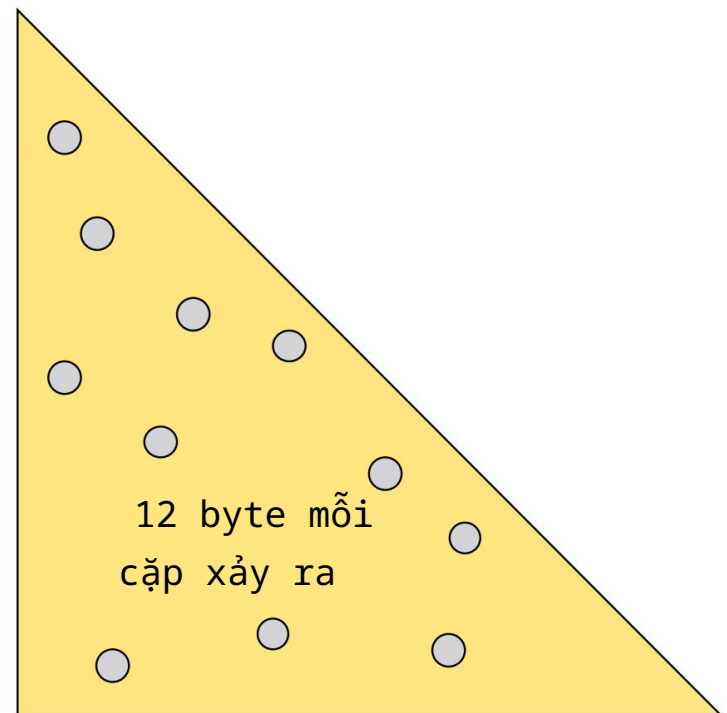
Nếu số nguyên và ID mục có kích thước 4 byte, chúng ta cần xấp xỉ 12 byte cho các cặp có số lượng > 0

Cộng thêm một số chi phí phát sinh cho bảng băm

Comparing the Two Approaches



Ma trận tam giác



Bộ ba (mục i, mục j, số lượng)

Comparing the Two Approaches

Phương pháp 1: Ma trận tam giác

n = tổng số mặt hàng

Chỉ đếm cặp phần tử $\{i, j\}$ nếu $i < j$

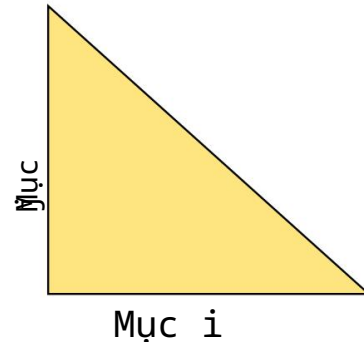
Giữ số lượng cặp theo thứ tự từ điển:

$\{1,2\}, \{1,3\}, \dots, \{1,n\}, \{2,3\}, \{2,4\}, \dots, \{2,n\}, \{3,4\}, \dots$

Cặp $\{i, j\}$ ở vị trí: $[n(n-1) - (n-i)(n-i+1)]/2 + (j-i)$

Tổng số cặp $n(n-1)/2$; tổng số byte = $O(n^2)$

Ma trận tam giác yêu cầu 4 byte cho mỗi cặp



Phương pháp 2 sử dụng 12 byte cho mỗi cặp xuất hiện
(nhưng chỉ áp dụng cho các cặp có số lượng > 0)

Tiếp cận 2 nhíp. Tiếp cận 1 nếu ít hơn $1/3$.
Các cặp khả thi thực sự xảy ra

Comparing the Two Approaches

Phương pháp 1: Ma trận tam giác

n = tổng số mặt hàng

Ch

Vấn đề phát sinh khi chúng ta có quá

$\{1,2\}, \{1,3\}, \dots, \{1,n\}, \{2,3\}, \{2,4\}, \dots, \{2,n\}, \{3,4\}, \dots$

nhiều mục đến nỗi không thể lưu

$[n(n-1) - (n-i)(n-i+1)]/2 + (j-i)$

trữ hết tất cả các cặp vào bộ nhớ.

Ma trận tam giác

Phương pháp 2 Chúng ta có thể làm tốt hơn không?

(nh

Tiếp cận 2 nhíp. Tiếp cận 1 nếu ít hơn $1/3$.

Các cặp khả thi thực sự xảy ra

A-Priori Algorithm

Các khái niệm

- chính:
- Tính đơn điệu của “thường xuyên”
 - Khái niệm về các cặp ứng viên
 - Mở rộng cho các tập hợp mục lớn hơn

A-Priori Algorithm – (1)

Một phương pháp tiếp cận hai bước được gọi là **A-Priori** giới hạn nhu cầu về bộ nhớ chính.

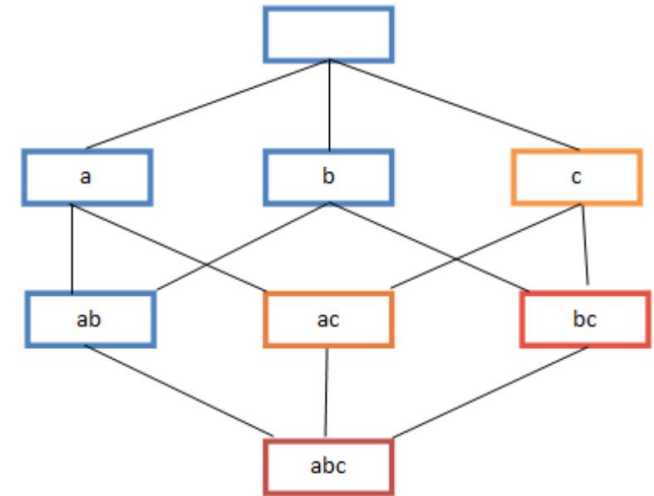
Ý tưởng chính: tính đơn điệu

Nếu một tập hợp các mục I xuất hiện ít nhất

s lần, thì mọi tập hợp con J của I cũng xuất hiện ít nhất s lần.

Mệnh đề đảo ngược cho các cặp:

Nếu mục i không xuất hiện trong giỏ s , thì không có cặp nào bao gồm i có thể xuất hiện trong giỏ s .



Vậy, A-Priori tìm các cặp tần số như thế nào?

A-Priori Algorithm – (2)

Lượt 1: Đọc các giỏ hàng và đếm trong bộ nhớ chính số lần xuất hiện của từng **mục riêng lẻ**

Chỉ yêu cầu bộ nhớ tỷ lệ thuận với số lượng mục

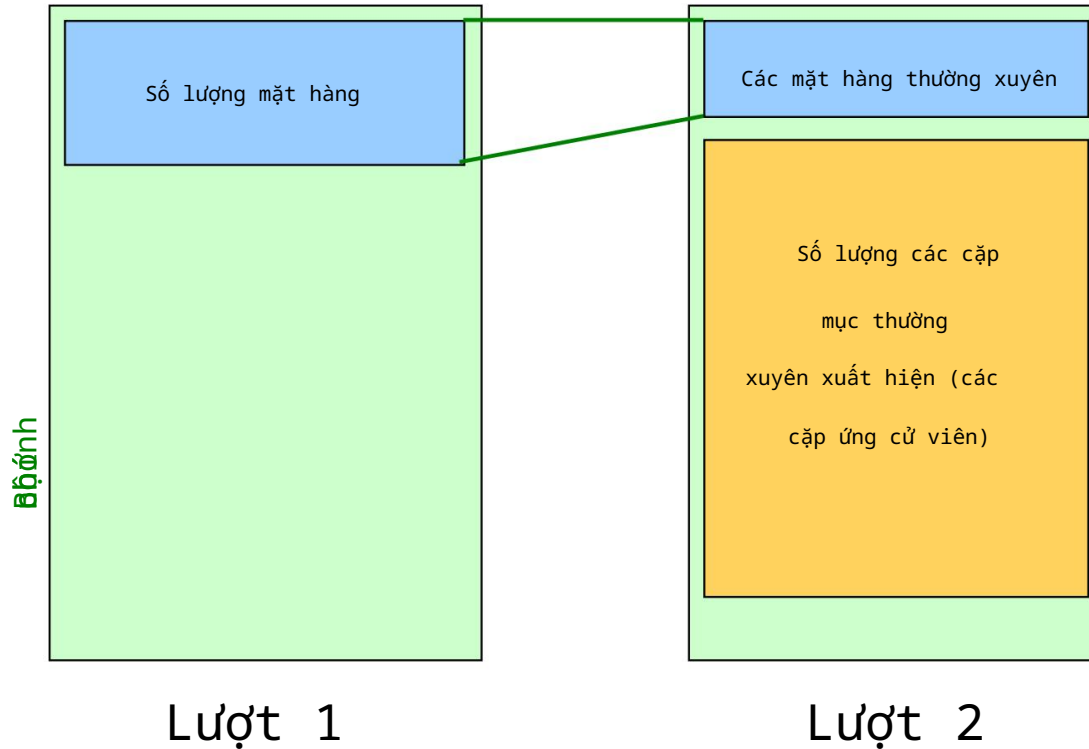
Các mục xuất hiện $\geq$ lần là các mục thường xuyên.

Bước 2: Đọc lại các giỏ hàng và ghi lại số lượng chỉ những cặp mà cả hai phần tử đều xuất hiện thường xuyên (từ Bước 1).

Yêu cầu bộ nhớ (để đếm) tỷ lệ thuận với bình phương của Số lượng các mục **xuất hiện thường xuyên** (không phải bình phương tổng số mục)

Thêm vào đó là danh sách các mặt hàng thường xuyên xuất hiện (để bạn biết những mặt hàng nào cần được tính)

Main-Memory: Picture of A-Priori



Ô màu xanh lá cây biểu thị dung lượng bộ nhớ chính khả dụng. Các ô nhỏ hơn biểu thị cách bộ nhớ được sử dụng.

Detail for A-Priori

Bạn có thể sử dụng

phương pháp ma trận

tam giác với n = số lượng mục

thường xuyên

Có thể tiết kiệm không gian so
với việc lưu trữ các bộ ba.

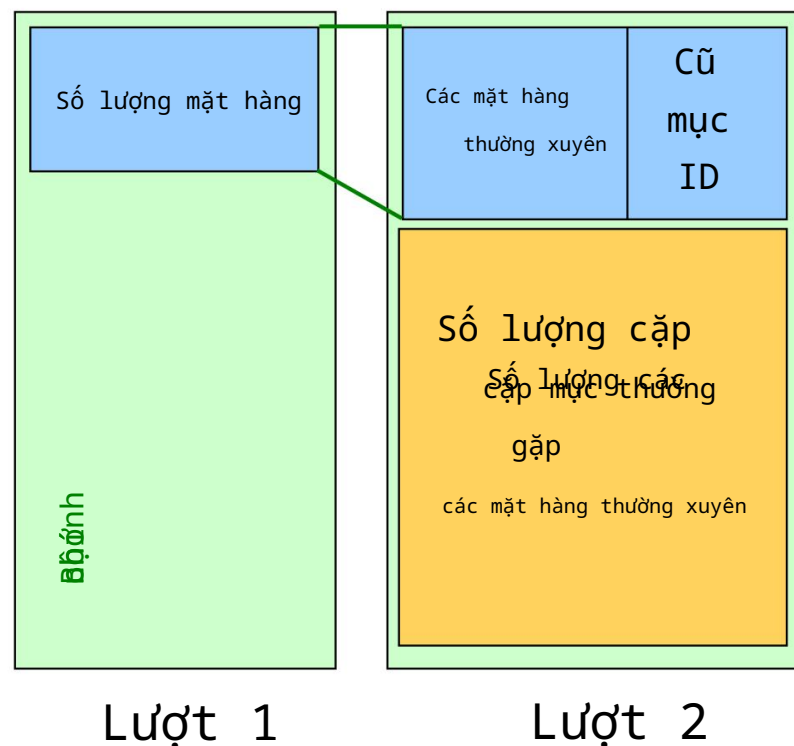
Mẹo : đánh số lại

các mặt hàng thường xuyên

xuất hiện 1, 2, . và lập bảng

liên hệ số mới với số mặt hàng

ban đầu.



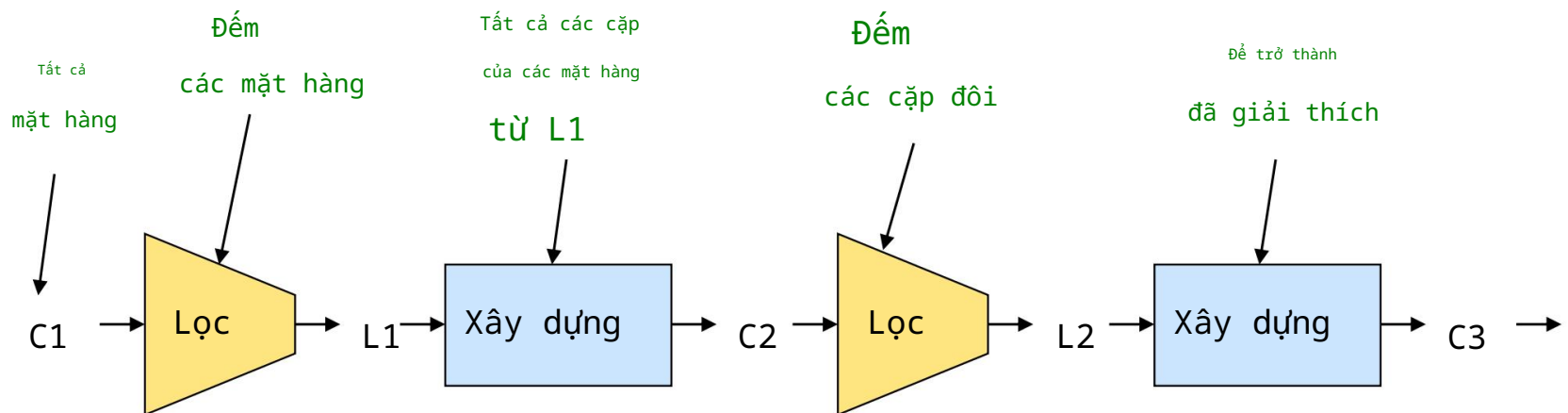
Frequent Triples, Etc.

Với mỗi k , chúng ta xây dựng hai tập hợp

bộ k phần tử (tập hợp có kích thước k):

C_k = bộ k ứng viên = những bộ có thể là tập hợp thường xuyên (hỗ trợ $> \underline{s}$) dựa trên thông tin từ lượt cho $k-1$

L_k = tập hợp các bộ k phần tử thực sự thường xuyên



Example

** Lưu ý rằng ở đây chúng ta tạo ra các ứng viên mới bằng cách tạo ra C_k từ L_{k-1} và L_1 .
Nhưng người ta có thể cẩn thận hơn với việc tạo ra các ứng viên. Ví dụ, trong C_3 , ta biết $\{b, m, j\}$ không thể là tập hợp thường xuyên vì $\{m, j\}$ không phải là tập hợp thường xuyên.

Các bước giả định của thuật toán A-Priori

$C_1 = \{ \{b\} \{c\} \{j\} \{m\} \{n\} \{p\} \}$

Đếm số lần xuất hiện của các tập mục trong

C_1 Loại bỏ các mục không thường xuyên. Ta được: $L_1 = \{ b, c, j, m \}$

Tạo $C_2 = \{ \{b, c\} \{b, j\} \{b, m\} \{c, j\} \{c, m\} \{j, m\} \}$

Đếm số lần xuất hiện của các tập mục trong

C_2 Loại bỏ các mục không thường xuyên. $L_2 = \{ \{b, m\} \{b, c\} \{c, m\} \{c, j\} \}$

Tạo $C_3 = \{ \{b, c, m\} \{b, c, j\} \{b, m, j\} \{c, m, j\} \}$

**

Đếm số lần xuất hiện của các tập mục trong C_3

Loại bỏ các mục không thường xuyên. $L_3 = \{ \{b, c, m\} \}$

A-Priori for All Frequent Itemsets

Một lượt xử lý cho mỗi k (kích thước tập hợp mục)

Cần có đủ dung lượng trong bộ nhớ chính để đếm từng bộ

k -tuple ứng viên.

Đối với dữ liệu giỏ hàng điển hình và hợp lý

hỗ trợ (ví dụ: 1%), $k = 2$ yêu cầu bộ nhớ nhiều nhất

Nhiều khả năng mở rộng khác nhau:

Các quy tắc liên kết với khoảng thời gian:

Ví dụ: Đàn ông trên 65 tuổi có 2 xe ô tô

Các quy tắc liên kết khi các mục nằm trong một hệ thống phân loại

Bánh mì, bơ Mứt trái cây

Bánh nướng, Sản phẩm từ sữa Thực phẩm đóng hộp

Giảm mức hỗ trợ khi tập hợp mục lớn hơn.

PCY (Park-Chen-Yu) Algorithm

Khái niệm chính: •

Cải tiến thuật toán A-Priori • Khai

thác bộ nhớ trống trong lần xử lý đầu tiên • Sử dụng

các nhóm dữ liệu thường xuyên

PCY (Park-Chen-Yu) Algorithm

Quan sát:

Trong lượt 1 của thuật toán A-Priori, hầu hết bộ nhớ đều ở trạng thái nhàn rỗi.

Chúng tôi chỉ lưu trữ số lượng từng mặt hàng riêng lẻ.

Chúng ta có thể sử dụng bộ nhớ nhàn rỗi để giảm lượng bộ nhớ cần thiết trong bước 2 không?

Bước 1 của PCY: Ngoài việc đếm số lượng mục, hãy duy trì một bảng băm với số lượng ngăn/phần tử nhiều nhất có thể trong bộ nhớ. Giữ một bộ đếm cho mỗi ngăn mà các cặp mục được băm vào

Ghi chú:

Xô Giỏ

Với mỗi xô, chỉ cần ghi số lượng, không cần ghi giá trị thực tế. ghép các cặp băm đó vào thùng!

PCY Algorithm – First Pass

CHO (mỗi giỏ):

CHO (mỗi mặt hàng trong giỏ hàng):

cộng thêm 1 vào số lượng của mặt hàng;

Mới
trong
PCY



Đối với (mỗi cặp sản phẩm trong giỏ):

chia đôi cặp đó vào một cái xô;

cộng thêm 1 vào số lượng của thùng đó;

Observations about Buckets

Quan sát: Nếu một nhóm chứa một cặp thường xuyên xuất hiện,

Vậy thì chắc chắn xô nước sẽ thường xuyên được sử dụng.

Tuy nhiên, ngay cả khi không có cặp thường xuyên nào, một nhóm vẫn có thể được coi là nhóm thường xuyên.

Vì vậy, chúng ta không thể sử dụng hàm băm để loại bỏ bất kỳ thành viên (cặp) nào của một nhóm “thường xuyên”.

Nhưng, đối với một nhóm có tổng số lượng nhỏ hơn s , thì không có cặp nào trong nhóm đó có thể là cặp thường xuyên

Các cặp có mã băm đến nhóm này có thể bị loại bỏ vì ứng viên (ngay cả khi cặp đó gồm 2 mục thường xuyên xuất hiện)

Lượt 2:

Chỉ tính các cặp có băm đến các nhóm thường xuyên.

PCY Algorithm – Between Passes

Thay thế các nhóm dữ liệu bằng một **vectơ bit**:

1 có nghĩa là số lượng thùng chứa đã vượt quá số lượng thùng hỗ trợ .

(gọi đó là **nhóm thường xuyên**); 0 nghĩa là không

Các giá trị đếm số nguyên 4 byte được thay thế bằng bit, do đó vectơ bit chỉ cần 1/32 bộ nhớ.

Ngoài ra, hãy xác định những mục nào thường xuyên xuất hiện và liệt kê chúng cho lần kiểm tra thứ hai.

PCY Algorithm – Pass 2

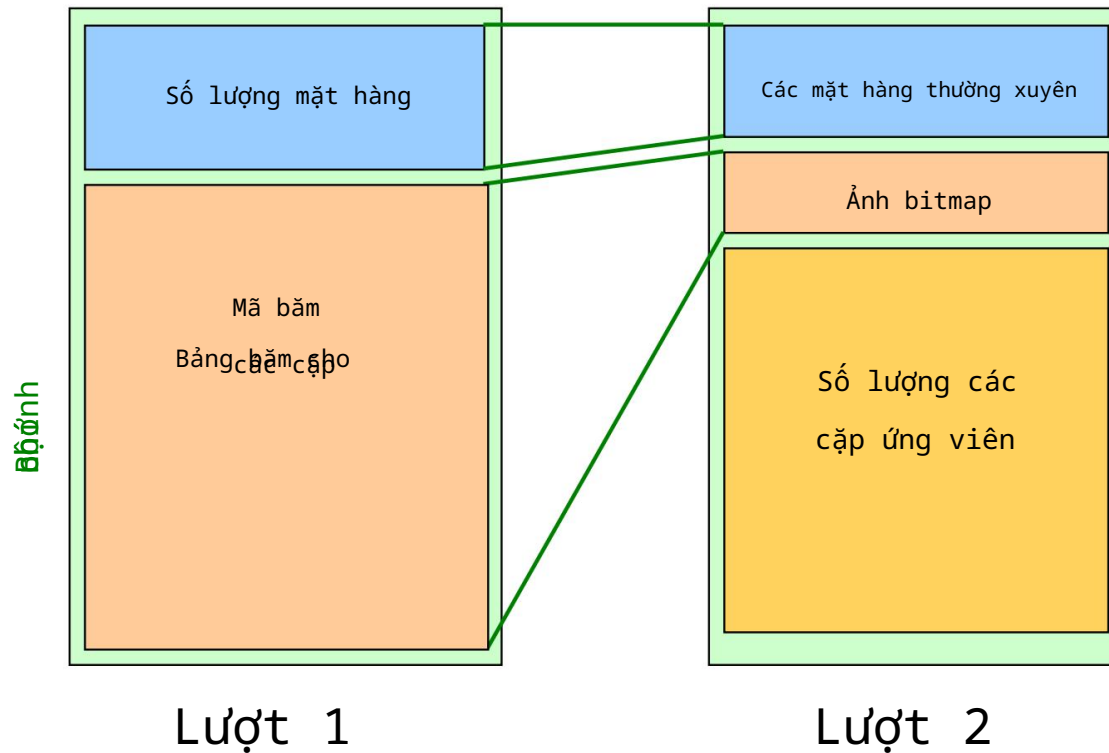
Đếm tất cả các cặp $\{i, j\}$ thỏa mãn điều kiện sau:

Điều kiện để trở thành một **cặp ứng viên**:

1. Cả i và j đều là các mục thường xuyên xuất hiện.
2. Cặp $\{i, j\}$ được băm thành một nhóm có bit trong vectơ bit là 1 (tức là nhóm thường xuyên).

Cả hai điều kiện đều cần thiết cho
có khả năng trở nên thường xuyên.

Main-Memory: Picture of PCY



More Extensions to A-Priori

Cuốn sách MMDS đề cập đến một số phần mở rộng khác.

Vượt ra ngoài ý tưởng PCY: “Đa giai đoạn” và “Multihash”

Để tự đọc thêm, hãy tham khảo Mục 6.4 của MMDS.

Video được đề xuất (bắt đầu từ khoảng 10:10):

<https://www.youtube.com/watch?v=AGAkNiQnbjY>

Frequent Itemsets in ≤ 2 Passes

Các khái niệm

chính: • Thuật toán lấy mẫu ngẫu

nhiên • Thuật toán Savasere-Omiecinski-Navathe (SON)

• Thuật toán Toivonen

Frequent Itemsets in ≤ 2 Passes

Các thuật toán A-Priori, PCY, v.v., thực hiện k lượt để tìm các tập hợp mục thường xuyên có kích thước k

Chúng ta có thể giảm số lần chuyển bóng được không?

Sử dụng 2 lượt hoặc ít hơn cho tất cả các kích cỡ.

nhưng có thể bỏ sót một số tập hợp mục thường gặp.

3 cách tiếp cận khác nhau:

1. Lấy mẫu ngẫu nhiên
2. S_{ON} (Savasere, Omiecinski và Navathe)
3. Toivonen

Random Sampling (1)

Lấy mẫu ngẫu nhiên các giỏ hàng trong chợ.

Chạy a-priori hoặc một trong những cải tiến của nó trong bộ nhớ chính:

Vì vậy, chúng ta không phải trả phí cho mỗi lần truy cập/ghi dữ liệu ổ đĩa.

thời điểm chúng ta tăng kích thước của các tập hợp mục

Giảm ngưỡng hỗ trợ một cách tương ứng

để phù hợp với kích

thước mẫu.

Ví dụ: nếu mẫu của bạn là $1/100$ số giỏ hàng, hãy sử dụng $s/100$ làm ngưỡng hỗ trợ thay vì s .

Bộ nhớ

Bản sao của
vật mẫu
giỏ

Không gian
vì
số lượng

Random Sampling (2)

Để tránh kết quả dương tính giả: Tùy chọn, xác minh lại xem các cặp ứng viên có thực sự xuất hiện thường xuyên trong toàn bộ tập dữ liệu hay không bằng cách thực hiện lần quét thứ hai.

Nhưng bạn sẽ không bắt được những tập hợp xuất hiện thường xuyên trong toàn bộ dữ liệu nhưng không xuất hiện trong mẫu.

Ngưỡng nhỏ hơn, ví dụ: $s/125$, giúp phát hiện nhiều tập hợp mục thực sự thường xuyên hơn.

Nhưng cần nhiều không gian hơn

Thuật toán SON cố gắng giải quyết vấn đề này (tiếp theo)

SON Algorithm – 2-pass algorithm

Thuật toán SON: Đọc lặp đi lặp lại các tập con nhỏ của giỏ hàng vào bộ nhớ chính và chạy thuật toán trong bộ nhớ để tìm tất cả các tập hợp mục thường xuyên xuất hiện.

Chúng tôi sử dụng làm ngưỡng nếu mỗi đoạn là một phần nhỏ của toàn bộ tập và là ngưỡng hỗ trợ.

Lưu ý: chúng tôi không lấy mẫu mà xử lý toàn bộ mẫu.

tập được chia thành các khối có kích thước bằng bộ nhớ

Một tập hợp các mục trở thành **ứng viên** nếu nó được tìm thấy xuất hiện thường xuyên trong **một hoặc nhiều** tập con của các giỏ hàng.

SON Algorithm – 2nd pass

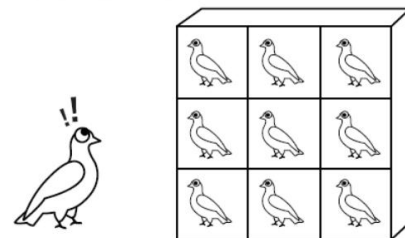
Lần thứ hai, hãy đếm tất cả các ứng viên.

lập các tập hợp mục và xác định mục nào xuất hiện thường xuyên trong toàn bộ tập hợp.

Ý tưởng chính về “tính đơn điệu”: Chúng tôi đặt ngưỡng hỗ trợ cho mỗi khối sao cho một tập hợp mục không thể xuất hiện thường xuyên trong toàn bộ tập dữ liệu trừ khi nó xuất hiện thường xuyên trong ít nhất một tập hợp con.

Nguyên tắc chuồng bồ câu

THE PIGEONHOLE PRINCIPLE



Toivonen's Algorithm: Intro

Lượt 1:

Bắt đầu với một mẫu ngẫu nhiên, nhưng hạ thấp ngưỡng chọn mẫu một chút:

Ví dụ: Nếu mẫu chiếm 1% số giỏ hàng, hãy sử dụng $s/125$ làm ngưỡng hỗ trợ thay vì $s/100$.

Tìm các tập hợp mục thường xuyên xuất hiện trong mẫu

Thêm **đường viền âm** vào các tập hợp mục xuất hiện thường xuyên trong mẫu:

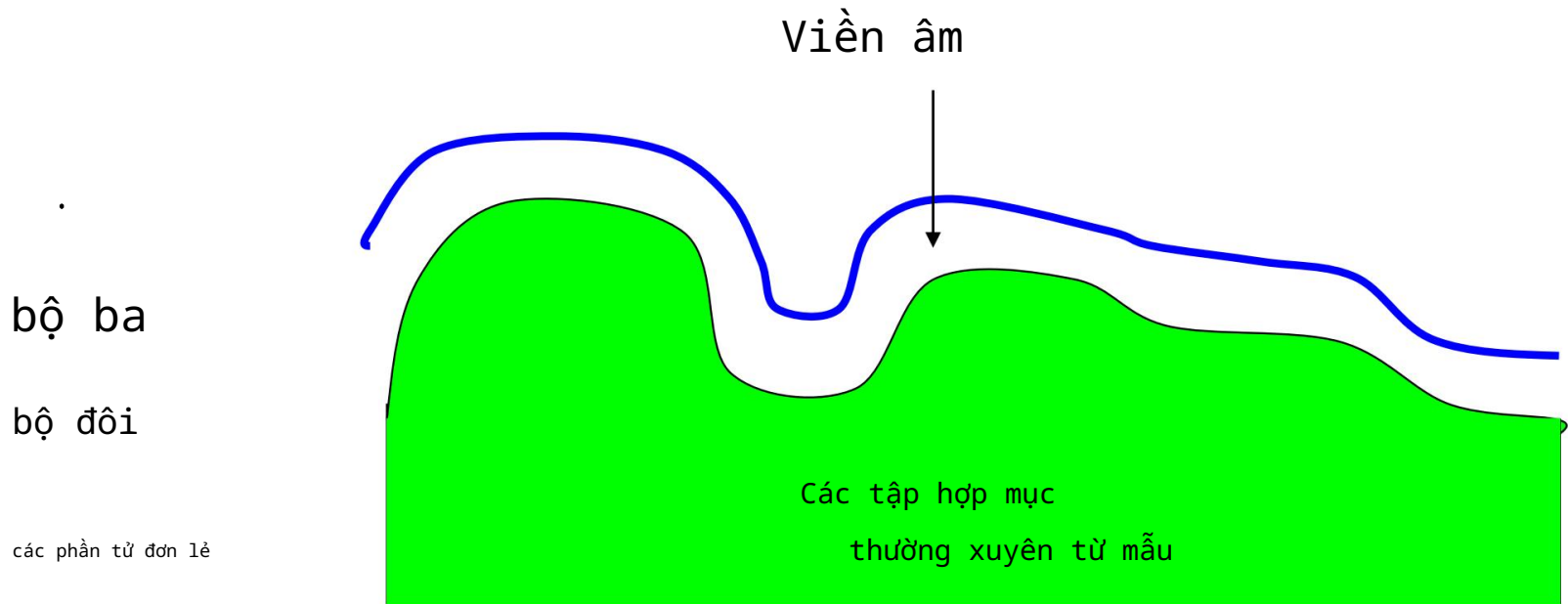
Biên âm: Một tập hợp mục nằm ở giá trị âm.

ranh giới nếu nó **không** thường xuyên xuất hiện trong mẫu, nhưng **tất cả** các tập con trực tiếp của nó đều là

Tập con trực tiếp = “xóa chính xác một phần tử”

Example: Negative Border

$\{A,B,C,D\}$ nằm trong ranh giới âm nếu và chỉ nếu: 1. Nó không xuất hiện thường xuyên trong mẫu, nhưng 2. Tất cả các phần tử $\{A,B,C\}$, $\{B,C,D\}$, $\{A,C,D\}$ và $\{A,B,D\}$ đều xuất hiện thường xuyên.



Toivonen's Algorithm

Lượt 1:

Bắt đầu với mẫu ngẫu nhiên, nhưng hạ thấp ngưỡng một chút cho tập con.

Thêm **biên âm** của các tập hợp mục xuất hiện thường xuyên trong mẫu.

Lượt 2:

Đếm tất cả các tập hợp mục thường xuyên ứng cử viên từ lượt đầu tiên, và cũng đếm các tập hợp nằm trong ranh giới âm của chúng.

Nếu không có tập hợp mục nào từ biên âm xuất hiện thường xuyên, thì chúng ta đã tìm thấy **tất cả** các tập hợp mục thường xuyên.

Điều gì sẽ xảy ra nếu chúng ta phát hiện ra điều gì đó ở vùng biên âm thường xuyên?

Chúng ta phải bắt đầu lại với một mẫu khác!

Cố gắng chọn ngưỡng hỗ trợ sao cho xác suất lỗi thấp, đồng thời số lượng tập hợp mục được kiểm tra trong lần chạy thứ hai vẫn nằm trong bộ nhớ chính.

If Something in the Negative Border Is Frequent . . .



Theorem:

Nếu có một tập hợp mục S xuất hiện thường xuyên trong dữ liệu đầy đủ nhưng không xuất hiện thường xuyên trong mẫu, thì ranh giới âm chứa ít nhất một tập hợp mục xuất hiện thường xuyên trong dữ liệu đầy đủ.

Chứng minh bằng phương pháp phản chứng:

Giả sử không phải vậy; tức là;

1. Có một tập hợp mục S xuất hiện thường xuyên trong toàn bộ dữ liệu nhưng không xuất hiện thường xuyên trong dữ liệu đầy đủ.

thường xuyên xuất hiện trong mẫu và

2. Không có gì ở phần viền âm xuất hiện thường xuyên trong dữ liệu đầy đủ.

Gọi T là tập con **nhỏ nhất** của S mà không xuất hiện thường xuyên trong tập hợp con đó.

mẫu (nhưng mọi tập con của T đều là)

T xuất hiện thường xuyên trong toàn bộ (S xuất hiện thường xuyên + tính đơn điệu).

Nhưng sau đó T lại nằm ở ranh giới âm (mâu thuẫn)